

Unit testing

Automated tests are especially useful for debugging if they test as little code as possible. This helps to isolate bugs, because a test can only fail if the bug is within the code used by that test. A “unit” is the smallest testable amount of code, so this testing strategy is called “unit testing”.

A unit could be a single method, or several methods, or a whole class, or possibly multiple classes. This all depends on how the software is designed — for example, each test in our `VectorTest` class only tested one method. This was possible because the `Vector` class’s methods don’t call each other, and they don’t use any other classes.

However, we did need to create a `Vector` object for the tests, so technically we also tested the constructor. Similarly, it would be impossible to test a setter method without also calling the getter method to check the result.

In another example, a `Rectangle` object has two `Vector` objects, and the rectangle’s methods calls some of the vector’s methods. So, the `Rectangle` class cannot be tested in isolation from the `Vector` class.

A “**unit test suite**” is a collection of unit tests, grouped into one or more test classes. Typically, each group tests one class, or a small number of collaborating classes.

Principles of unit testing

A unit test should:

- Test what the software is *supposed* to do, regardless of what it *actually* does.
- Create as few objects and call as few methods as possible in order to test the behaviour in question.
- Use simple data where the correct results are known.
- Not depend on anything which can’t be controlled, like the current system time, network connectivity, or random numbers.
- Be short, and run quickly.
- Not pass “by accident”.
- Be independent of other unit tests. In particular, the result of a test should never depend on whether or not other tests have been run already.

A unit test suite should:

- Test the behaviour with both valid and invalid data. If a method should respond to invalid data in a particular way, test that it does respond that way.
- Have *distinct* unit tests, which test different behaviours or use cases. There is no benefit if three unit tests either all pass or all fail together, telling you the same thing three times.

- Test as much of the software as reasonably possible. This can be measured by “code coverage”, i.e. what percentage of the code is reached when the test suite is run.
- If the software has a specification, then the unit test suite should test as much of the *specification* as possible.

Choosing test cases

A “**test case**” means the input data used in a test, the action to be performed with that data, and the expected result. For example, here are the test cases we used to test the Vector class:

Input	Action	Expected result
new Vector(3, 4)	call getX method	It returns 3.
new Vector(3, 4)	call getY method	It returns 4.
new Vector(3, 4)	call length method	It returns 5.0.

To choose an “action to be performed”, think of a way your code can be used. This usually means calling one method. However, if a method should change an object’s state, we must call at least one other method afterwards to test that the state changed correctly.

Alternatively, we may need to call other methods beforehand to put an object into a state where some behaviour can be tested; for example, to test that a vending machine vends a candy bar, you have to put some coins in first.

Therefore, the “action” can have up to three parts:

1. Any steps needed to prepare the test.
2. The steps you actually want to test.
3. Any steps needed to check that the result is correct.

The test inputs should be simple inputs with known correct results. For all but the simplest actions, it’s usually a good idea to have multiple test cases with different inputs for the same action. Good test inputs include:

- Typical but small values, e.g. small numbers or short strings.
- Empty values, e.g. the number 0, the empty string "", or null as appropriate.
- Invalid inputs, e.g. -1 if the input is required to be non-negative.
- Boundary values: if the method should do different things for numbers above or below 10, then test 9, 10 and 11 as separate cases.
- If multiple values are needed for the same test case, such as 3 and 4 in the Vector tests, make these different. For example, if the getX method incorrectly returns 4, then you will know where that incorrect value came from.

You should always work out the “expected result” yourself, by thinking about it — not by running the code. If you’re testing the code, you can’t assume that the result it *does* give is the correct one! This is why it’s best to choose simple inputs, where the correct result is easy to work out.